Computational Complexity

Focus on algorithms for decidable languages.

We are interested in solving problems via efficient algorithms.

Efficiency can be measured in terms of a resource as a function of the input size, e.g., running time, memory / space utilized, energy requirement, amount of communication etc.

Input size can refer to number of bits (int multiplication), number of items in list (sorting), number of rows & columns (matrix multiplication).

Typically measure runtime as a **worst-case** function of input length.

Focus on scaling behaviour of algorithm, i.e., how does it behave **asymptotically** as a function of input size.

Asymptotic Notation

Let $f, g : \mathbb{N} \longrightarrow \mathbb{R}^+$

Def $f(n) \in O(g(n))$ if there exist constants c, n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for all} \quad n > n_0.$$

Informally, **Big-O** is an **upper** bound, i.e., $f(n)$ is **at most** proportional to $g(n)$.

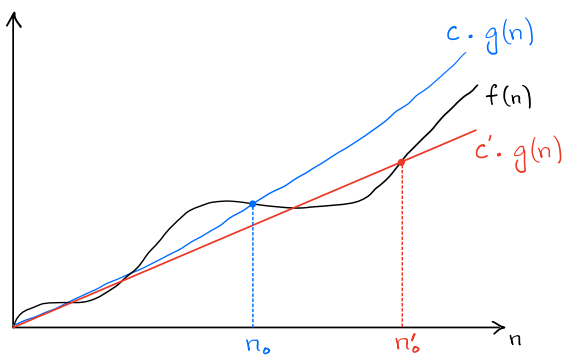
Def $f(n) \in \Omega(g(n))$ if there exist constants c', n_0' such that

$$f(n) \geq c' \cdot g(n) \quad \text{for all} \quad n > n_0'.$$

Informally, **Big-Ω** is a **lower** bound, i.e., $f(n)$ is **at least** proportional to $g(n)$.

Def $f(n) \in \Theta(g(n))$ if $f(n) \in O(g(n))$ & $f(n) \in \Omega(g(n))$

In other words, the same $g(n)$ is both an upper bound & a lower bound upto constant factors.



Big-O Relationship

$$1 \longrightarrow \lg(\lg^* n) \longrightarrow \lg^* n \longrightarrow \lg\lg n \longrightarrow \lg n \longrightarrow \sqrt{n} \longrightarrow n/\lg n \longrightarrow n \longrightarrow n\lg n$$

$$n\lg n \longrightarrow n^2 \longrightarrow n^3 \longrightarrow n^{100} \longrightarrow n^{\lg n} \longrightarrow 2^n \longrightarrow 3^n \longrightarrow n! \longrightarrow n^n$$

$$n^n \longrightarrow 2^{2^n} \longrightarrow 2^{2^{2^{\cdots^2}}} \Big\}\, n \text{ times}$$

Qs. Given $\langle M \rangle$, input x & int n, does $\langle M \rangle$ halt after at most n^3 steps on x.

Claim Any TM to solve this needs atleast n^3 steps.

Proof Assume a TM P does it in $n^{2.99}$. Produce a P' as follows on input $\langle M \rangle$ & n:

1. Run forever if M halts in at most n^3 steps.
2. Halt if M runs for more than n^3 steps.

→ Before we get into the self-recursive contradiction, note that P' halts here in at most $n^{2.99}$ steps.

Run $P'(P', n)$ to obtain contradiction.

This implies the Time Hierarchy Theorem, which implies that there is no faster way to predict a TM's behaviour, given a finite number of steps, other than to run it. So, given more time always results in a TM able to solve more problems.

Search for Optimal Algorithms

For a given problem, a key question is to find the fastest algorithm for it.

E.g.,

(OPE) integer multiplication

- Standard algorithm is $O(n^2)$
- Karatsuba $O(n^{\lg 3})$
- Conjecture $O(n \lg n)$ is optimal.

matrix multiplication

- Standard algorithm is $O(n^3)$
- Strassen $O(n^{2.78})$
- Williams $O(n^{2.371339})$
- Conjecture For all $\varepsilon > 0$, there exists an $O(n^{2+\varepsilon})$ algorithm
- AlphaTensor discovered a better algorithm for 4x4 matrices

Blum's Speedup Theorem

We can construct problems such that there is provably no best algorithm.

There exists a problem, such that for all $O(f(n))$ algorithms, there also exists $O(\lg(f(n)))$ algorithm.

Def Let $f : \mathbb{N} \longrightarrow \mathbb{R}^+$

$TIME(f(n))$ is the collection of all languages that are decidable in $O(f(n))$ steps by a single tape DTM.

Similar definition for $SPACE(f(n))$.

Theorem Let $f(n) \geq n$. Every $f(n)$ time multitape TM has an equivalent $O(f^2(n))$ single tape TM.

Proof Exercise.

Def P is the class of languages decidable in polynomial time by a single tape DTM.

$$P = \bigcup_k TIME(n^k)$$

Strong / Extended Church Turing Thesis

All reasonable models of step-counting algorithms are polynomially equivalent.
i.e., can simulate each other with a polynomial overhead.

Example Problems in P

Longest increasing subsequence
Stable marriage
Gaussian elimination
Linear programming
Primality testing
2-coloring a map
Maximum matching
& many more

Not Known to be in P

3-coloring a map
Max Clique
Travelling salesman
Protein folding
& many more important problems

Def Runtime of an NTM N is the max number of steps that N uses in any branch on input length n.

Theorem Let $f(n) \geq n$. Every $f(n)$ time NTM has an equivalent $2^{O(f(n))}$ DTM.

Proof Exercise.